

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Clasificación y Reconocimiento</i>				
NÚMERO DE PRÁCTICA:	09	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	07/07/26	HORA DE PRESENTACIÓN	23:00		
INTEGRANTE (s): Venero Guevara Christian Henry				NOTA:	
DOCENTE(s): <i>MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS GitHub: https://github.com/Cvenero/Laboratorios_Computacion-Grafica/tree/main/Laboratorio_09 1. Introducción Para el desarrollo de la presente práctica de laboratorio se seleccionó el proyecto "Pong Game using Hand Gestures", el cual consiste en la implementación de un videojuego clásico de Pong controlado mediante el reconocimiento de gestos de las manos a través de una cámara web. Este proyecto permite aplicar conceptos de visión por computadora, procesamiento de imágenes en tiempo real y detección de landmarks de manos, utilizando las bibliotecas OpenCV y cvzone sobre el lenguaje Python. El interés de elegir este proyecto radica en la posibilidad de interactuar con un sistema informático sin necesidad de dispositivos físicos de entrada como mouse o teclado, reemplazándolos por el movimiento natural de las manos frente a la cámara. Esto permite reforzar el entendimiento práctico de temas como la detección de manos mediante modelos de aprendizaje automático, el cálculo de colisiones en un entorno de coordenadas de imagen, y el manejo de superposición de imágenes con canal alfa (PNG transparente) sobre un fotograma de video. A lo largo del desarrollo se identificaron y solucionaron distintos problemas: la correcta diferenciación de la mano izquierda y derecha, la implementación de la física de rebote de la pelota, la definición de las condiciones de finalización del juego (Game Over), y finalmente la

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

incorporación de un fondo personalizado que solo muestra las manos del jugador, ocultando el resto de la imagen capturada por la cámara.

2. OBJETIVOS

2.1. Objetivo general

Implementar un videojuego de Pong controlado mediante el reconocimiento de gestos de manos, utilizando visión por computadora en tiempo real.

2.2. Objetivos específicos

- Detectar y diferenciar correctamente la mano izquierda y la mano derecha del usuario frente a la cámara.
- Asociar cada mano detectada a un bate (paleta) del juego, superponiendo una imagen PNG con transparencia sobre el video en tiempo real.
- Implementar la lógica de movimiento y rebote de la pelota, incluyendo colisión con los bates y con los bordes superior e inferior del campo de juego.
- Definir las condiciones de finalización de una ronda (Game Over) cuando la pelota sale del campo sin ser interceptada por un bate.

3. MARCO TEÓRICO

3.1. Visión por computadora (Computer Vision)

Es un campo de la inteligencia artificial que permite a los sistemas informáticos interpretar y comprender el contenido visual de imágenes o videos. En este proyecto se emplea para capturar el video de la cámara web y procesarlo en tiempo real.

3.2. OpenCV

Es una biblioteca de código abierto orientada al procesamiento de imágenes y video. En este proyecto se utiliza para capturar el video, redimensionar fotogramas, dibujar figuras geométricas, superponer texto y aplicar transformaciones como el volteo horizontal (flip) de la imagen.

3.3. cvzone

Es una biblioteca construida sobre OpenCV y MediaPipe que simplifica tareas comunes de visión por computadora, como la detección de manos, el conteo de dedos levantados y la superposición de imágenes PNG con canal alfa sobre un fotograma de video.

3.4. Detección de manos y landmarks

La detección de manos se basa en un modelo de aprendizaje profundo que identifica 21 puntos de referencia (landmarks) en cada mano detectada, correspondientes a las

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

articulaciones de los dedos y la palma. Estos puntos permiten calcular la posición de dedos específicos, así como determinar si un dedo se encuentra levantado o no.

3.5. Envoltente convexa (Convex Hull)

Es un algoritmo geométrico que, dado un conjunto de puntos, calcula el polígono más pequeño que los contiene a todos. En este proyecto se utiliza para generar el contorno de la mano a partir de los 21 landmarks detectados, con el fin de crear una máscara visual.

3.6. Superposición con canal alfa (Overlay PNG)

Es la técnica utilizada para colocar una imagen con transparencia (como los bates o la pelota) sobre un fotograma de video, respetando las zonas transparentes de la imagen original.

4. DESARROLLO

4.1. Detección y clasificación de manos

El primer paso del sistema consiste en capturar el video de la cámara y detectar las manos presentes en cada fotograma mediante HandDetector. Inicialmente se presentó un problema en el cual solo se detectaba una mano por frame, ya que la clasificación original de "Left" y "Right" entregada por la biblioteca no correspondía con la posición real en pantalla, debido al volteo horizontal de la imagen aplicado previamente.

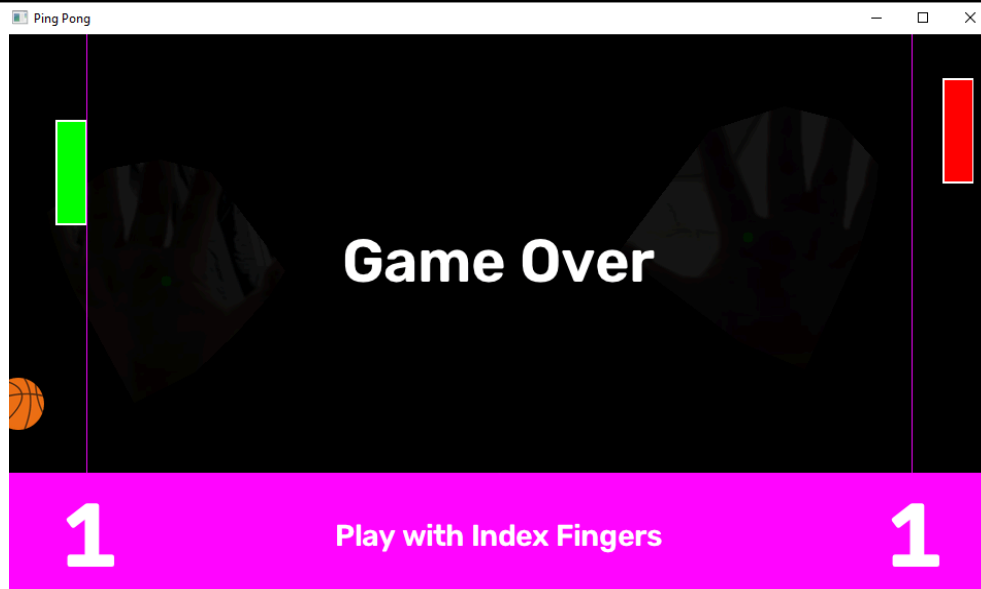
La solución consistió en reasignar el tipo de cada mano según su posición horizontal (coordenada X) respecto al centro del fotograma, en lugar de depender de la etiqueta interna del detector:

Python

```
for hand in handsT:
    lmList = hand["lmList"]
    cx = hand["center"][0]
    hand["type"] = "Left" if cx < FrameWidth // 2 else "Right"
```

```
hands = handsT
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4



De esta manera, la mano que aparece en la mitad izquierda de la pantalla siempre controla el bate izquierdo (verde), y la mano en la mitad derecha controla el bate derecho (rojo), independientemente de la clasificación interna del modelo.

4.2. Superposición de los bates

Una vez clasificada correctamente cada mano, la función `updateBat` se encarga de posicionar la imagen del bate correspondiente sobre la posición vertical del dedo índice del usuario, restringiendo el movimiento para que no sobrepase los límites verticales de la pantalla:

Python

```
def updateBat(img, idxPos, type_hand):
    x_pos = 50 if type_hand == "Left" else 910
    y_pos = idxPos[1] - 60
    y_pos = max(0, min(y_pos, 540 - 120))
    bat_img = bat_left if type_hand == "Left" else bat_right
    img = overlayPNG(img, bat_img, [x_pos, y_pos])
    return img, y_pos
```

Cada bate se mantiene fijo en un carril horizontal ($x = 50$ para la izquierda, $x = 910$ para la derecha), mientras que su posición vertical se actualiza en cada fotograma según el movimiento de la mano.

4.3. Movimiento y colisión de la pelota

Uno de los principales problemas identificados fue que la pelota rebotaba indefinidamente contra los bordes laterales de la ventana, sin que existiera una

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

condición que determinara la pérdida de la ronda. Originalmente solo existía la lógica de rebote contra el bate y contra los bordes superior e inferior:

Python

```
# Left Bat hit
if (ballPosX < boardWidth) and (leftBatY < ballPosY < leftBatY + 120):
    ballSpeedX = abs(ballSpeedX)
    ballPosX += 30

# Right Bat hit
elif (ballPosX + ballWH > FrameWidth - boardWidth) and (rightBatY < ballPosY < rightBatY + 120):
    ballSpeedX = -abs(ballSpeedX)
    ballPosX -= 30
```

Se incorporó una nueva condición que detecta cuándo la pelota sale completamente del campo de juego por alguno de los dos lados, otorgando el punto al jugador contrario y marcando el fin de la ronda:

Python

```
# Ball out on the Left side (Right player scores)
if ballPosX < 0:
    rightPoint += 1
    gameOver = True

# Ball out on the Right side (Left player scores)
elif ballPosX + ballWH > FrameWidth:
    leftPoint += 1
    gameOver = True
```

1

Play with Index Fingers

1

Esta validación se ubicó inmediatamente después de la verificación de colisión con los bates, y antes del bloque encargado de reiniciar la partida, garantizando que el sistema reaccione en el mismo fotograma en que la pelota sale del campo.

4.4. Reinicio de partida y pantalla de Game Over

Cuando se cumple alguna de las condiciones de salida descritas anteriormente, el sistema reinicia la posición de la pelota al centro del campo, genera una nueva dirección

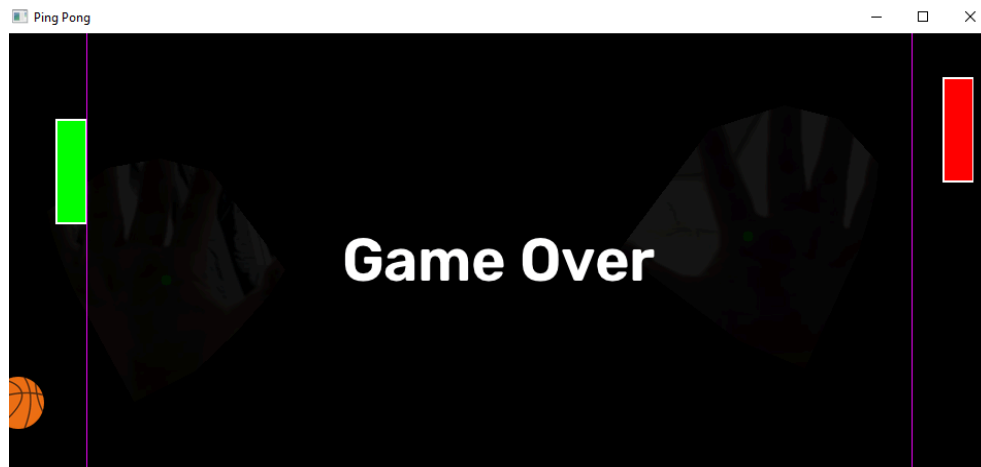
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

aleatoria de movimiento y detiene momentáneamente el juego para mostrar el mensaje "Game Over" en pantalla durante dos segundos:

Python

```
if (gameOver):
    gameStarted = False
    gameOver = False
    ballSpeedX = random.choice([ballSpeed, -ballSpeed])
    ballSpeedY = random.choice([ballSpeed, -ballSpeed])
    ballPosX, ballPosY = (FrameWidth // 2) - (ballWH // 2), (boardMaxHeight // 2) - (ballWH // 2)

    imgG0 = addTextToCenter(img, "Game Over", fontScale=2, color=(255, 255, 255),
thickness=3)
    cv2.imshow(windowName, imgG0)
    cv2.waitKey(2000)
```



4.5. Ocultamiento del fondo real mediante máscara de manos

Como mejora final, se buscó que únicamente las manos del usuario fueran visibles en pantalla, ocultando el rostro y el resto del entorno capturado por la cámara. Para ello se implementó la función `maskOnlyHands`, la cual genera una máscara binaria a partir de la envolvente convexa (convex hull) de los 21 landmarks de cada mano detectada, y reemplaza todo lo que se encuentre fuera de dicha envolvente por una imagen de fondo personalizada:

Python

```
def maskOnlyHands(img, hands, background):
    mask = np.zeros(img.shape[:2], dtype=np.uint8)

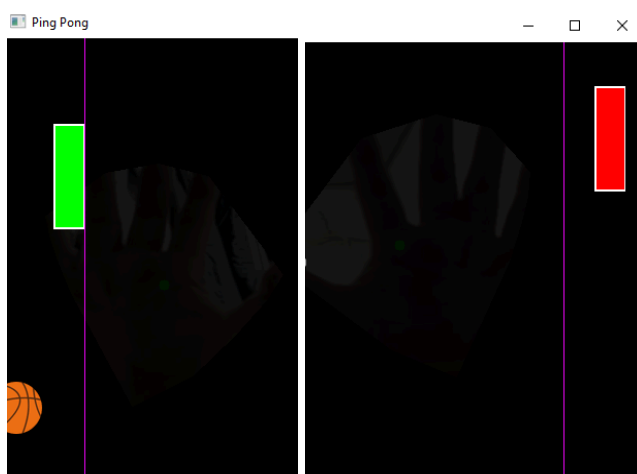
    for hand in hands:
        lmList = hand["lmList"]
        points = np.array([[p[0], p[1]] for p in lmList], dtype=np.int32)
        hull = cv2.convexHull(points)
        cv2.fillConvexPoly(mask, hull, 255)

    mask3 = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
    result = np.where(mask3 == 255, img, background)
    return result
```

Esta función se invoca dentro del bucle principal, inmediatamente después de obtener la lista de manos detectadas y antes de dibujar el tablero del juego, de modo que el fondo se reemplace antes de superponer los elementos del juego (bates, pelota, marcador):

Python

```
img = maskOnlyHands(img, hands, background)
img = updateBoard(img)
```



5. CONCLUSIONES

- Se logró implementar correctamente la detección y diferenciación de ambas manos del usuario, corrigiendo el problema inicial de clasificación mediante el uso de la posición horizontal real de cada mano en pantalla, en lugar de depender exclusivamente de la etiqueta interna del modelo de detección.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

- La incorporación de la validación de salida de la pelota permitió completar la lógica esencial del juego de Pong, estableciendo un sistema de puntuación funcional y una condición clara de finalización de ronda.
- El uso de la envolvente convexa sobre los landmarks de la mano resultó ser una solución eficiente para generar una máscara visual que oculta cualquier parte del cuerpo distinta a las manos, sin necesidad de recurrir a modelos de segmentación más complejos o costosos en procesamiento.
- El desarrollo del proyecto permitió reforzar conceptos prácticos de visión por computadora en tiempo real, procesamiento de imágenes con OpenCV, y el manejo de bibliotecas especializadas como cvzone para la detección de manos.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] OpenCV, "OpenCV documentation," OpenCV.org. [Online]. Available: <https://docs.opencv.org/>. [Accessed: Jul. 7, 2026].
- [2] Computer Vision Zone, "cvzone: A Computer Vision package that makes it easy to run Image processing and AI functions," GitHub. [Online]. Available: <https://github.com/cvzone/cvzone>. [Accessed: Jul. 7, 2026].
- [3] Google AI Edge, "Hand landmarks detection guide," Google for Developers. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker. [Accessed: Jul. 8, 2026].
- [4] Python Software Foundation, "Python 3 documentation," Python.org. [Online]. Available: <https://docs.python.org/3/>. [Accessed: Jul. 8, 2026].
- [5] NumPy Developers, "NumPy documentation," NumPy.org. [Online]. Available: <https://numpy.org/doc/>. [Accessed: Jul. 8, 2026].